

คู่มือโปรแกรมเมอร์

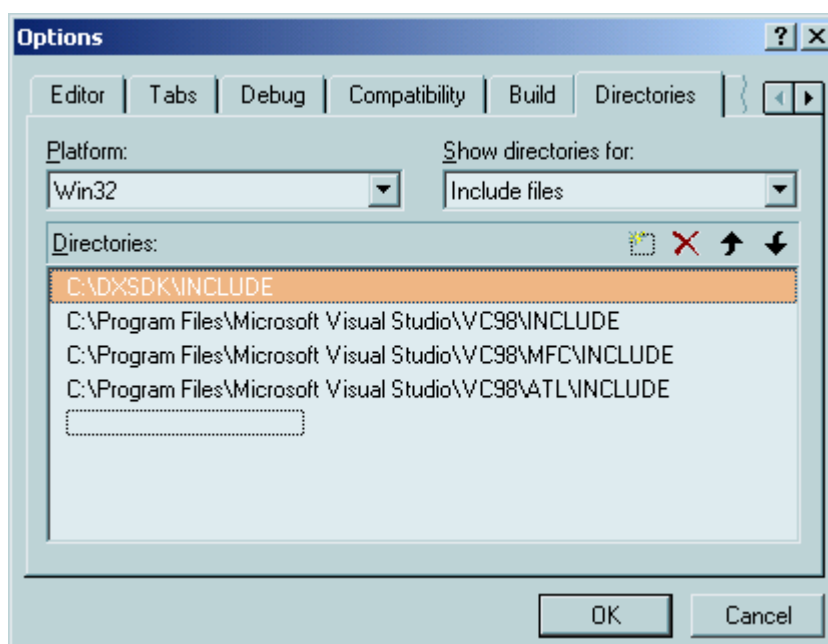
โปรแกรมที่ใช้ในแคมปัสเสมือนจริงนั้น จะมีอยู่ 2 โปรแกรมด้วยกัน คือ

1. โปรแกรมฝั่งเซิร์ฟเวอร์ ทำหน้าที่เป็นศูนย์กลางในการรับข้อมูลจากไคลเอนต์ต่างๆ เพื่อประมวลผล และส่งออกไปยังไคลเอนต์ต่างๆ
2. โปรแกรมฝั่งไคลเอนต์ เป็นส่วนของโปรแกรมเครื่องลูกข่ายที่ให้ผู้ใช้งานได้ใช้งานเพื่อการเล่น และส่งอีเมลล์ โดยผลิตเพลินไปกับการแสดงผลภาพ 3 มิติ

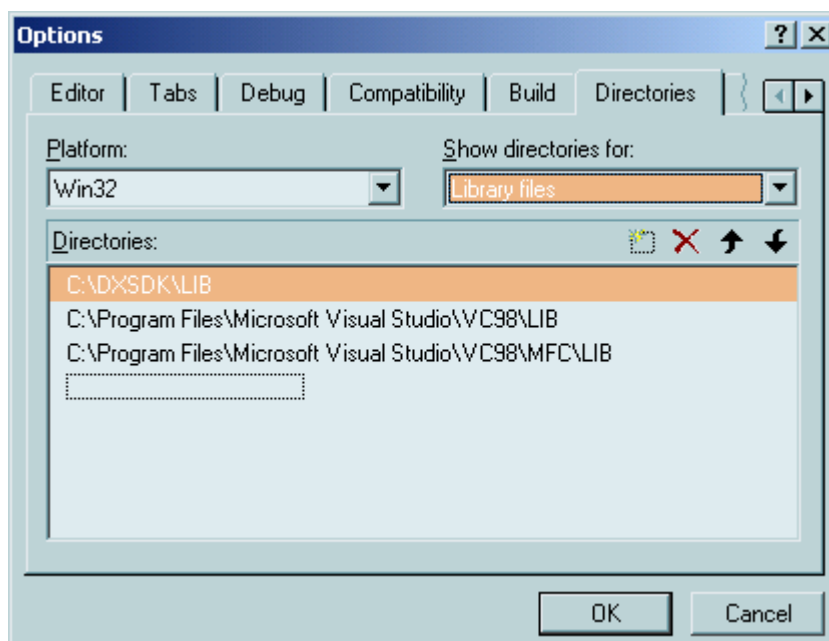
1. โปรแกรมฝั่งเซิร์ฟเวอร์

การเตรียม Microsoft Visual C++ ให้พร้อมใช้งานกับ DirectX

การใช้งาน DirectX SDK ร่วมกับคอมไพเลอร์ Microsoft Visual C++ นั้นจะต้องกำหนดไคลเรททอรีที่เก็บไฟล์เฮดเดอร์ (header file) และไฟล์ไลบรารีก่อน โดยเลือก Option จากเมนู Tool แล้วเลือกแท็บ Directories ซึ่งจะปรากฏไดอะล็อกบ็อกแล้วกำหนดค่าดังรูป



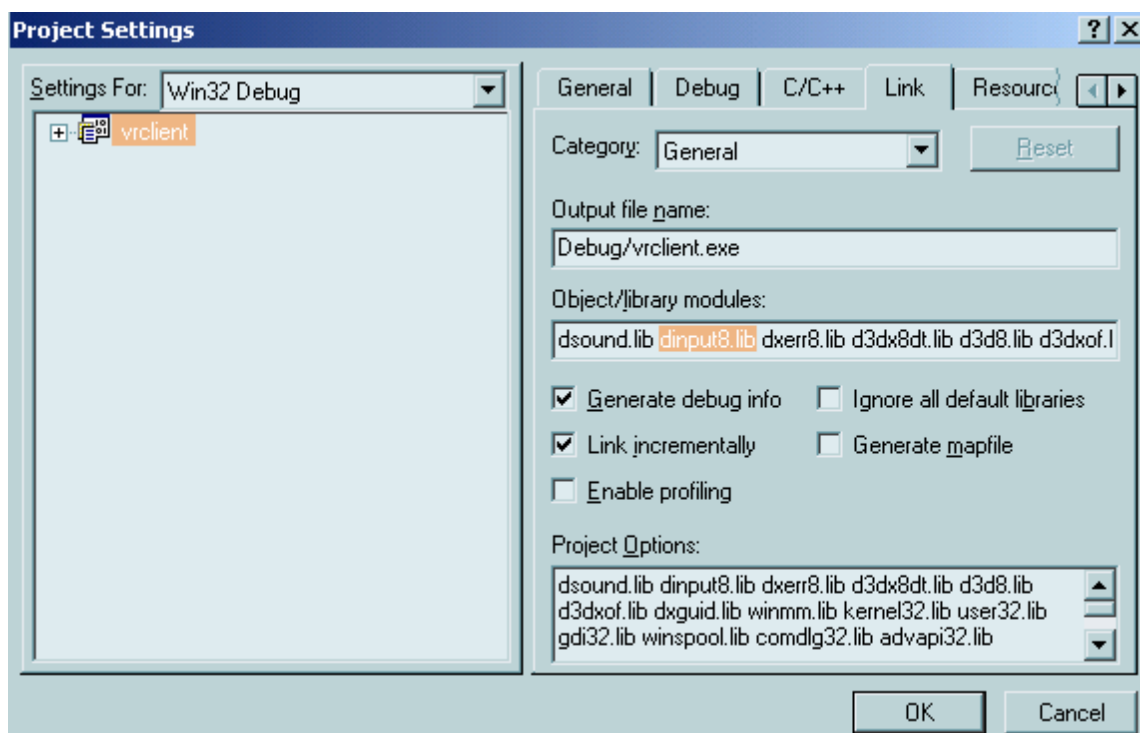
รูปที่ 1 ไดอะล็อกบ็อกสำหรับการกำหนดไคลเรททอรีของเฮดเดอร์ไฟล์ในไคลเร็กเอ็กซ์



รูปที่ 2 ใดอะลอกบ็อกสำหรับการกำหนดไคเรกทอรีของไลบรารีไฟล์ในไคเร็กเอ็กซ์

เลื่อนตำแหน่งไคเรกทอรีไปอยู่ด้านบนสุดเนื่องจากไลบรารีที่มากับตัวคอมไพเลอร์อาจจะเก่ากว่าเวอร์ชันที่ต้องการใช้งาน

นอกจากนี้ยังต้องกำหนดไลบรารีที่จะให้คอมไพเลอร์ลิงก์อีกด้วยโดยเลือก setting จากเมนู Project จะปรากฏใดอะลอกบ็อกแล้วจึงกำหนดค่าไลบรารีที่จะใช้เพิ่มเข้าไป ดังรูป



รูปที่ 3 ใดอะลอกบ็อกสำหรับการกำหนดไลบรารีที่จะนำมาลิงก์ในขณะคอมไพล์

```

-----Configuration: vrserver - Win32 Debug-----
Compiling...
vrserver.cpp
Linking...
vrsmtp.obj : error LNK2001: unresolved external symbol _WSACleanup@0
vrsmtp.obj : error LNK2001: unresolved external symbol _WSAGetLastError@0
vrsmtp.obj : error LNK2001: unresolved external symbol _WSAStartup@8
vrsmtp.obj : error LNK2001: unresolved external symbol _recv@16
vrsmtp.obj : error LNK2001: unresolved external symbol _send@16
vrsmtp.obj : error LNK2001: unresolved external symbol _connect@12
vrsmtp.obj : error LNK2001: unresolved external symbol _htons@4
vrsmtp.obj : error LNK2001: unresolved external symbol _getservbyname@8
vrsmtp.obj : error LNK2001: unresolved external symbol _socket@12
vrsmtp.obj : error LNK2001: unresolved external symbol _gethostbyname@4
vrsmtp.obj : error LNK2001: unresolved external symbol _inet_ntoa@4
vrsmtp.obj : error LNK2001: unresolved external symbol _gethostname@8
Debug/vrserver.exe : fatal error LNK1120: 12 unresolved externals
Error executing link.exe.

vrserver.exe - 13 error(s), 0 warning(s)

```

รูปที่ 4 ตัวอย่างข้อความผิดพลาดในการคอมไพล์โดยที่ไม่ได้กำหนดคอมไพล์ลิงค์

คอมไพล์ลิงค์ไลบรารีที่จำเป็นของฝั่งเซิร์ฟเวอร์

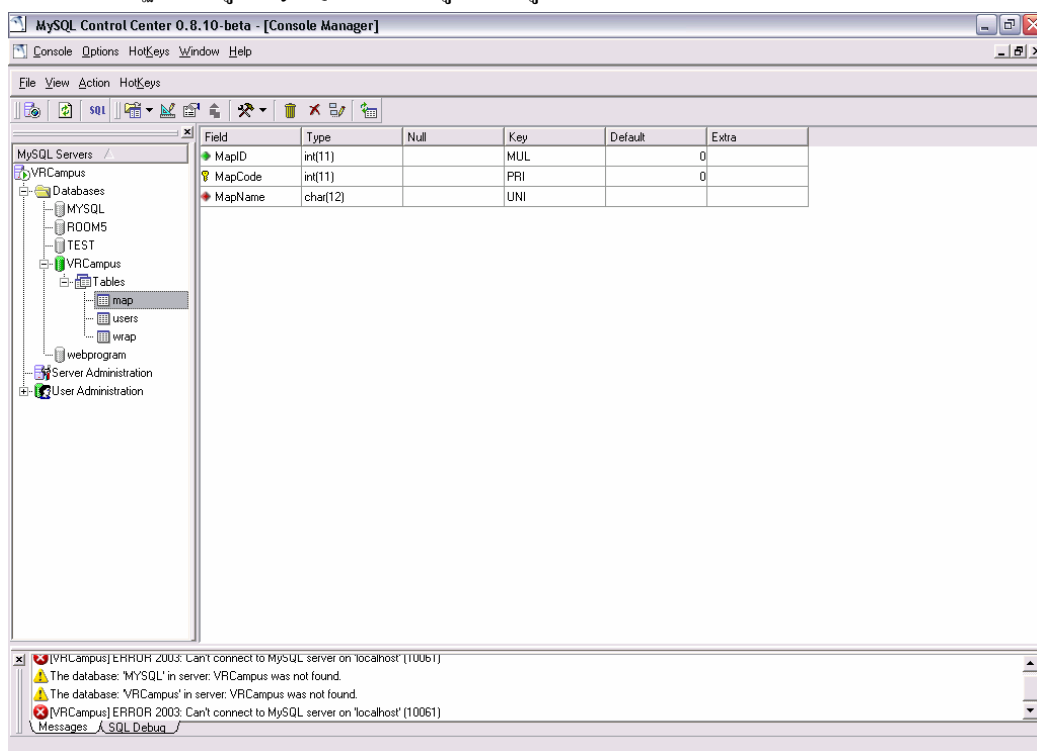
- dplay.lib
- dxguid.lib
- ws2_32.lib

คอมไพล์ไลบรารีที่จำเป็นของฝั่งไคลเอ็นท์

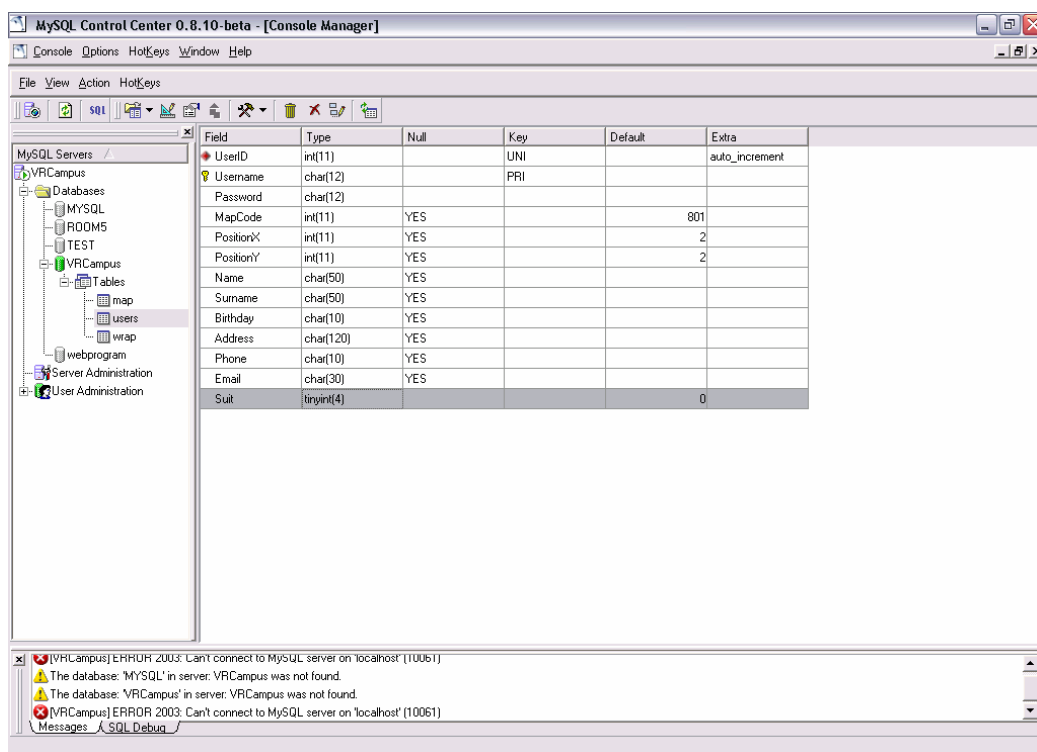
- dsound.lib
- dinput8.lib
- dxerr8.lib
- d3dx8dt.lib
- d3d8.lib
- d3dxof.lib
- dxguid.lib

ฐานข้อมูลที่ใช้ จะมีตารางอยู่ด้วยกัน 3 ตาราง คือ ตาราง map, users และ wrap ซึ่งมี Field ต่างๆ ในแต่ละตาราง ตามรูปที่ 5-7 ซึ่งก่อนที่จะเปิดเซิร์ฟเวอร์ จะต้องสร้างตารางเหล่านี้เสียก่อน หรือสามารถ copy โฟลเดอร์ \Data\VRCampus ซึ่งจะมีไฟล์ฐานข้อมูลอยู่ เข้าไปไว้ในโฟลเดอร์ MySQL ได้เลย ซึ่งตามรูปข้างล่างนี้ ใช้โปรแกรม MySQL Control Center ซึ่งมีการใช้งานง่าย หน้าตา Interface คล้ายๆ MS SQL Server

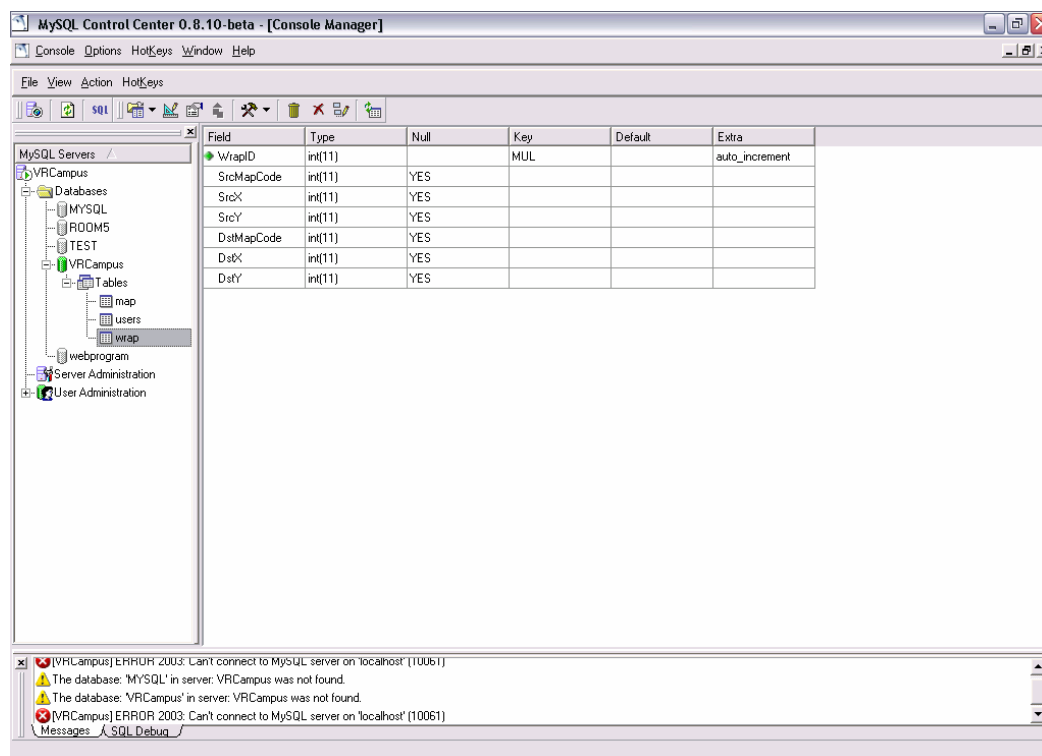
สำหรับการเปิดฐานข้อมูล MySQL ให้ทำงานนั้น สามารถวิธีติดตั้งและใช้งานได้จากคู่มือติดตั้งและใช้งาน รวมถึงการติดตั้งและใช้งาน โปรแกรม MySQL-Front และ MySQL Control Center ซึ่งเป็นเครื่องมือจัดการฐานข้อมูล MySQL สามารถดูได้จากคู่มือติดตั้งและใช้งาน



รูปที่ 5 แสดง Map Table



รูปที่ 6 แสดง Users Table



รูปที่ 7 แสดง Wrap Table

โปรแกรมฝั่งเซิร์ฟเวอร์ จะต้องมีการติดต่อกับฐานข้อมูล MySQL ต้องใช้ lib และ include ไฟล์ เพื่อติดต่อเข้าฐานข้อมูล MySQL ซึ่งสามารถโหลดได้จาก <http://mysql.oregonstate.edu/Downloads/mysql++/mysql++-1.7.1-1-win32-vc++.zip> โดยมีวิธีดังนี้

1. นำไฟล์ mysql++.lib ซึ่งอยู่ใน \lib นำมาใส่ไว้ใน Library Directory ของ Visual C++
2. นำ header file ซึ่งจะอยู่ใน \include, \mysql\include ทั้งหมด ไปใส่ไว้ใน Include Directory ของ Visual C++
3. นำ libmysql.lib อยู่ใน \mysql\lib นำไปใส่ใน Library Directory ของ Visual C++
4. ไฟล์ libmysql.dll อยู่ใน \mysql\lib จะต้องนำไปใส่ไว้ใน Directory เดียวกับ Project ที่ต้องการติดต่อฐานข้อมูล และเมื่อเขียนโปรแกรมเสร็จแล้ว จะต้องนำ libmysql.dll ติดกับโปรแกรมไปด้วยทุกครั้ง ที่แจกจ่าย เพราะโปรแกรมจะทำงานไม่ได้หากขาด libmysql.dll

ในซอร์สโคดของโปรแกรมฝั่งเซิร์ฟเวอร์ จะมีโครงสร้าง Structure ต่างๆ ดังนี้

struct MAIL_INFO	// เป็นโครงสร้างที่เก็บข้อมูลของอีเมลที่จะส่ง // อยู่ในไฟล์ \VRCampus\VRMySQL\vrmysql.h
struct DB_USER_INFO	// เป็นโครงสร้างที่เก็บค่าต่างๆเหมือนตาราง Users // อยู่ในไฟล์ \VRCampus\VRMySQL\vrmysql.h
struct DB_MAP_INFO	// เป็นโครงสร้างที่เก็บค่าต่างๆเหมือนตาราง Map // อยู่ในไฟล์ \VRCampus\VRMySQL\vrmysql.h

struct DB_WRAP_INFO	// เป็นโครงสร้างที่เก็บค่าต่างๆเหมือนตาราง Wrap // อยู่ในไฟล์ \VRCampus\VRMySQL\vrmysql.h
struct APP_PLAYER_INFO	// เป็นโครงสร้างที่เก็บค่าของผู้ใช้เมื่อล็อกอินเข้ามา // อยู่ในไฟล์ \VRCampus\VRServer\vrserver.cpp
struct CMap	// เป็นโครงสร้างที่เก็บข้อมูลรหัสและชื่อแผนที่ // อยู่ในไฟล์ \VRCampus\VRServer\vrserver.h
struct CNpc	// เป็นโครงสร้างที่เก็บข้อมูล NPC ซึ่งมีหลายแบบ // อยู่ในไฟล์ \VRCampus\VRServer\vrserver.h
struct CWrap	// เป็นโครงสร้างที่เก็บข้อมูลจุดวาง // อยู่ในไฟล์ \VRCampus\VRServer\vrserver.h

ในซอร์สโคดของโปรแกรมฝั่งเซิร์ฟเวอร์ ประกอบด้วยคลาสต่างๆ ดังนี้

● CVRDatabase class	// อยู่ในไฟล์ \VRCampus\VRMySQL\vrmysql.h และ \VRCampus\VRMySQL\vrmysql.cpp
class CVRDatabase	// เป็นคลาสที่จัดการเกี่ยวกับฐานข้อมูล VRCampus
{	
public:	
char m_szHost[256];	// เป็นตัวแปรที่เก็บ IP โฮสต์ที่ติดต่อฐานข้อมูล
char m_szUsername[256];	// เป็นตัวแปรที่เก็บ Username ที่จะติดต่อเข้า ฐานข้อมูลของโฮสต์ที่กำหนดไว้ใน m_szHost
char m_szPassword[256];	// เป็นตัวแปรที่เก็บ Password ที่จะติดต่อเข้า ฐานข้อมูลของโฮสต์ที่กำหนดไว้ใน m_szHost
char m_szDatabase[256];	// เป็นตัวแปรที่เก็บชื่อฐานข้อมูลที่จะเข้าติดต่อ
char m_szTable[256];	// เป็นตัวแปรที่เก็บชื่อตาราง (ยังไม่ได้ใช้งาน)
char m_szSQLQuery[256];	// เป็นตัวแปรที่เก็บคำสั่ง SQL Query และ Execute
MYSQL *m_pConnection;	// เป็นตัวแปรพื้นฐานการติดต่อ ตัวแปรนี้จะถูกใช้ ติดต่อกับฐานข้อมูล และเมื่อ run Query ที่ได้รับมา จะเก็บค่าที่คืนมาเอาไว้ จนกว่าจะมีตัวอื่นมารับไป
MYSQL_RES *m_resultSet;	// เป็นตัวแปรที่เก็บค่าหลังจากส่งคำสั่ง SQL Query ไป Select ข้อมูลออกมา โดยจะรับมาจาก ตัวแปร m_pConnection อีกทีหนึ่ง

```
CVRDatabase( char* szHost,char* szUsername,char* szPassword,char* szDatabase );
```

```
// เป็น Constructor ของ class นี้ จะมีการกำหนด
ชื่อ Host, Username, Password, Database โดย
กำหนดให้เท่ากับตัวแปรที่มีค่าตายตัวไว้
```

```
~CVRDatabase() { if ( m_pConnection != NULL ) Close(); };
```

```
// เป็น Destructor ของ class นี้ จะมีการตรวจสอบ
ว่ามีการเชื่อมต่อกับฐานข้อมูลหรือยัง ถ้าเชื่อมต่อ
แล้วให้สั่งคำสั่ง Close() เพื่อปิดการเชื่อมต่อกับ
ฐานข้อมูล โดย Destructor นี้จะถูกใช้เมื่อวัตถุที่
เป็นตัวแปรแบบ class นี้ ถูกทำลาย
```

```
int      Connect();
```

```
// เป็นฟังก์ชัน ที่ใช้สำหรับเชื่อมต่อกับฐานข้อมูล เข้า
ตาม ฐานข้อมูลของ Host ที่กำหนดไว้ในตัวแปร
m_szDatabase และ m_szHost โดยใช้ Username
และ Password ที่กำหนดไว้ในตัวแปร
m_szUsername และ m_szPassword
```

```
int      Close();
```

```
// เป็นฟังก์ชัน ที่ใช้สำหรับปิดการเชื่อมต่อกับ
ฐานข้อมูล เมื่อมีการเชื่อมต่อกับฐานข้อมูลแล้ว
```

```
int      SQLQuery( char* szSQLQuery );
```

```
// เป็นฟังก์ชัน ที่ใช้สำหรับทำตามคำสั่ง SQL
ที่ส่งไปในตัวแปร szSQLQuery
```

```
int      ChkAuthorize( char* szUsername, char* szPassword );
```

```
// เป็นฟังก์ชัน ที่ใช้สำหรับเช็ค Username และ
Password ว่าถูกต้องหรือไม่จาก ตาราง Users ใน
ฐานข้อมูล VRCampus โดยฟังก์ชันจะ return ค่า
ออกมาต่างๆ กันดังนี้
```

0 – Username กับ Password ถูกต้อง

1 – ไม่มีชื่อ Username ในตาราง Users

2 – Password ไม่ถูกต้อง

```
int      GetMapCode( char* szUsername );
```

```
// เป็นฟังก์ชัน ที่ใช้สำหรับหาค่ารหัส MapCode ที่
เป็นแผนที่ที่ตัวละคร szUsername อยู่ ค้นหาจาก
ตาราง Map
```

```
int      GetPosX( char* szUsername );
```

```

// เป็นฟังก์ชัน ที่ใช้สำหรับหาค่าตำแหน่งในแกน
X ของผู้ใช้ที่มีชื่อ szUsername โดยฟังก์ชัน จะ
return ตำแหน่งในแกน X ออกมา

```

```

int      GetPosY( char* szUsername );

// เป็นฟังก์ชัน ที่ใช้สำหรับหาค่าตำแหน่งในแกน
Y ของผู้ใช้ที่มีชื่อ szUsername โดยฟังก์ชัน จะ
return ตำแหน่งในแกน Y ออกมา

```

```

int      GetCountRow( char* szTable );

// เป็นฟังก์ชัน ที่ใช้หาจำนวนแถวที่มีอยู่ในตาราง
ชื่อ szTable โดยฟังก์ชันจะ return ค่าจำนวนแถว
ออกมา

```

```

DB_WRAP_INFO*      GetWrapInfo( int nID );

// เป็นฟังก์ชัน ที่ return ค่าออกมาเป็น Structure
DB_WRAP_INFO ซึ่งเป็นโครงสร้างที่มีตัวแปร
ตาม field column ในตาราง Wrap โดยจะ Select
ออกมาเฉพาะแถวที่มี WrapID = nID

```

```

DB_MAP_INFO*      GetMapInfo( int nID );

// เป็นฟังก์ชัน ที่ return ค่าออกมาเป็น Structure
DB_MAP_INFO ซึ่งเป็นโครงสร้างที่มีตัวแปรตาม
field column ในตาราง Map โดยจะ Select ออกมา
เฉพาะแถวที่มี MapID = nID

```

```

DB_USER_INFO*      GetUserInfo( char* szUsername );

// เป็นฟังก์ชัน ที่ return ค่าออกมาเป็น Structure
DB_USER_INFO ซึ่งเป็นโครงสร้างที่มีตัวแปร
ตาม field column ในตาราง Users โดยจะ Select
ออกมาเฉพาะแถวที่มี Username = szUsername

```

```

int      UpdatePosition( char* szUsername, int nX, int nY );

// เป็นฟังก์ชัน ที่ทำการเก็บบันทึกข้อมูลตำแหน่ง
X,Y ของตัวละครที่มีชื่อใน szUsername เก็บเข้า
ตาราง Users

```

```

int      UpdateMap( char* szUsername, UINT nMapCode );

```



```
// เป็นฟังก์ชัน ที่ทำการเก็บบันทึกรหัสแผนที่ที่ตัว
// ละครที่มีชื่อใน szUsername ยื่นอยู่ เก็บเข้าตาราง
// Users
```

```
};
```

โปรแกรมฝั่งเซิร์ฟเวอร์จะรับข้อมูลจากผู้ใช้ มาเก็บไว้ในฐานข้อมูล เช่นการเก็บตำแหน่งตัวละครของผู้ใช้ เมื่อมีการเปลี่ยนแปลงตำแหน่ง โปรแกรมฝั่งไคลเอนท์ จะส่งข้อมูลมายังเซิร์ฟเวอร์เพื่อเก็บบันทึกตำแหน่งใหม่ลงฐานข้อมูล

● CVRLayerWnd

```
// อยู่ในไฟล์ \VRCampus\Common\vrcommon.h
// และ \VRCampus\Common\vrcommon.cpp
class CVRLayerWnd
{
// เป็นคลาสที่ทำให้ไดอะล็อกซ์สามารถแสดงผล
// แบบ Transparent ได้

public:
    CVRLayerWnd();           // Constructor
    virtual ~CVRLayerWnd();  // Destructor
    LONG AddLayeredStyle(HWND hWnd);
    // กำหนดรูปแบบของหน้าต่างให้เป็นแบบโปร่งใส
    BOOL SetLayeredWindowAttributes(HWND hWnd, COLORREF crKey, BYTE
    bAlpha, DWORD dwFlags);
    // กำหนดค่าสีโปร่งใส และความหนาของสี
    BOOL SetTransparentPercentage(HWND hWnd, BYTE byPercentage);
    // กำหนดค่าเปอร์เซ็นต์ในการโปร่งใสของสี

private:
    typedef BOOL (WINAPI* lpfnSetLayeredWindowAttributes)(HWND hWnd,
    COLORREF crKey, BYTE bAlpha, DWORD
    dwFlags);
    HMODULE m_hDll;
};
```

● CVRMultilineText

```
// อยู่ในไฟล์ \VRCampus\Common\vrcommon.h
// และ \VRCampus\Common\vrcommon.cpp
```

class CVRMultilineText	// เป็นคลาสที่ทำหน้าที่ เก็บบันทึกค่าให้ Text Control แสดงข้อมูล Text ออกมาแบบ Multiline โดยจะมีการกำหนด MaxLine ของ Multiline ที่จะแสดง เพื่อให้เมื่อมีการรันแสดงข้อมูล Text มากๆ จะไม่แสดงทั้งหมด จะแสดงเพียงบรรทัด MaxLine หลังสุดเท่านั้น
{	
public :	
CVRMultilineText(int nNumLine, int nNumCharPerLine);	// Construtor โดยจะมีการกำหนดค่า m_nMaxLine และ m_nMaxCharPerLine ให้เท่ากับ nNumLine และ nNumCharPerLine ตามลำดับ เป็นการกำหนดจำนวนบรรทัดที่จะแสดงสูงสุดและจำนวนตัวอักษรมากสุดในหนึ่งบรรทัด
virtual ~CVRMultilineText();	// Destructor ทำลายตัวแปรพอยน์เตอร์ทุกตัวที่มีในคลาส
void AddString(char*);	// เป็นฟังก์ชันที่นำค่า String มาเก็บไว้ในตัวแปร m_aszMessageLog ซึ่งเป็นอาร์เรย์ 2 มิติของ char
char* Retrieve();	// เป็นฟังก์ชันที่นำค่า String ในแต่ละ Field จาก m_aszMessageLog มารวมต่อกันเป็น String เดียว คือ m_szMessageLog ซึ่งเป็นอาร์เรย์ของ char โดยในแต่ละ Field เปรียบได้เป็นหนึ่งบรรทัด ซึ่งจะถูกคั่นด้วย “\r\n” ซึ่งเป็นตัวอักษร enter
char **m_aszMessageLog;	// เป็นตัวแปรอาร์เรย์ 2 มิติ ที่เก็บค่าข้อมูลที่จะแสดงออก Text Control
char *m_szMessageLog;	// เป็นตัวแปรอาร์เรย์ มิติเดียว ซึ่งเปรียบเป็น String โดยค่านี้จะถูกนำไปใช้แสดงใน Text Control โดยตรง
int m_nCountLine;	// เป็นตัวแปรชี้ตำแหน่งแถว ที่จะเพิ่มข้อมูลเข้าไป
int m_nIndexStart;	// เป็นตัวแปรเก็บตำแหน่งแถวเริ่มต้น ใน m_aszMessageLog
int m_nIndexStop;	// เป็นตัวแปรเก็บตำแหน่งแถวสุดท้าย ใน m_aszMessageLog

```

const char* const    GetLocalHostIp();
                        // เป็นฟังก์ชันหาหมายเลข IP ของ localhost

```

```

const char* const    GetLocalHostname();
                        // เป็นฟังก์ชันหาชื่อ localhost name

```

```

const char* const    GetMessageBody();
                        // เป็นฟังก์ชัน return ค่าข้อความส่วน body ออก

```

```

const char* const    GetReplyTo();
                        // เป็นฟังก์ชัน return ชื่อที่จะต้องกลับ

```

```

const char* const    GetSenderEmail();
                        // เป็นฟังก์ชัน return ชื่ออีเมลของผู้ส่ง

```

```

const char* const    GetSenderName();
                        // เป็นฟังก์ชัน return ชื่อของผู้ส่ง

```

```

const char* const    GetSubject();
                        // เป็นฟังก์ชัน return หัวข้อเรื่องของอีเมล

```

```

const char* const    GetXMailer();
                        // เป็นฟังก์ชัน return ค่า m_pc_XMailer

```

```

bool    Send();
                        // เป็นฟังก์ชันในการส่งเมล

```

```

void    SetMessageBody(const char body[]);
                        // เป็นฟังก์ชันกำหนดค่าให้ข้อความส่วน body

```

```

void    SetSubject(const char subject[]);
                        // เป็นฟังก์ชันกำหนดหัวข้อเรื่องให้กับอีเมล

```

```

void    SetSenderName(const char name[]);
                        // เป็นฟังก์ชันกำหนดชื่อของผู้ส่ง

```

```

void    SetSenderEmail(const char email[]);
                        // เป็นฟังก์ชันกำหนดชื่ออีเมลของผู้ส่ง

```

```

void    SetReplyTo(const char replyto[]);
                        // เป็นฟังก์ชันกำหนดชื่อที่จะตอบกลับ

```

```

void    SetXMailer(const char xmailer[]);
                        // เป็นฟังก์ชันกำหนดค่า m_pc_XMailer

```

```

private:

```

```

class CRecipient      // เป็นคลาสที่เป็นเสมือนตัวแปรหนึ่งในคลาส
                        CVRsmtp ซึ่งเป็นคลาสที่เก็บข้อมูลอีเมลบัฟเฟอร์
                        ไว้ก่อนที่จะทำการส่ง

```

```

{
    public:
        CRecipient();
        CRecipient& operator=(const CRecipient& src);
        virtual ~CRecipient();
        char* GetEmail();
        void SetEmail(const char email[]);
    private:
        char *m_pcEmail;
};

bool    bCon;                // เป็นตัวแปรบ่งบอกสถานะ การเชื่อมต่อกับ
                             เซิร์ฟเวอร์ หาก bCon = True แสดงว่าเชื่อมต่ออยู่
                             หาก False แสดงว่าไม่ได้เชื่อมต่อ

char    m_cHostName[MAX_PATH];
                             // เป็นตัวแปรเก็บชื่อ Host

char*    m_pcFromEmail;      // เป็นตัวแปรเก็บชื่ออีเมลผู้ส่ง
char*    m_pcFromName;      // เป็นตัวแปรเก็บชื่อผู้ส่ง
char*    m_pcSubject;       // เป็นตัวแปรเก็บหัวข้อผู้ส่ง
char*    m_pcMsgBody;       // เป็นตัวแปรเก็บข้อความส่วน body
char*    m_pcXMailer;       // เป็นตัวแปรเก็บค่า Xmailer
char*    m_pcReplyTo;       // เป็นตัวแปรเก็บชื่ออีเมลที่ตอบกลับ
char*    m_pcIPAddr;        // เป็นตัวแปรเก็บหมายเลข IP ของ localhost

WSADATA    wsaData;         // เป็นตัวแปรของ Winsock เป็นตัวเปิด Socket
                             และ Startup การใช้งาน Winsock

SOCKET    hSocket;          // เป็นตัวแปรแบบ SOCKET ที่ใช้ในการส่งอีเมล

std::vector<CRecipient*> Recipients;
                             // เป็นตัวแปรแบบ Vector ซึ่งมี Type ตัวแปรข้าง
                             ในแบบคลาส CRecipient

std::vector<CRecipient*> CCRecipients;
                             // เป็นตัวแปรแบบ Vector ซึ่งมี Type ตัวแปรข้าง
                             ในแบบคลาส CRecipient

std::vector<CRecipient*> BCCRecipients;

```

```

// เป็นตัวแปรแบบ Vector ซึ่งมี Type ตัวแปรข้าง
// ในแบบคลาส CRecipient
char* _formatHeader(); // เป็นฟังก์ชันจัดรูปแบบค่าข้อมูล Header ของเมลล์
bool _formatMessage(); // เป็นฟังก์ชันจัดรูปแบบค่าข้อมูล body
SOCKET _connectServerSocket(const char* server, const unsigned short
port=NULL);
// เป็นฟังก์ชันที่ทำการเชื่อมต่อกับ SMTP Server
};

```

ฟังก์ชันหลักๆ ที่ใช้ในการทำงาน ซึ่งเป็นแบบ Global มีดังต่อไปนี้

```

void cfnCenterWindow(HWND hwnd); // เป็นฟังก์ชันทำให้ไอคอนล็อกช้อยู่กลางหน้าจอ
// อยู่ในไฟล์ \VRCampus\common\vrcommon.cpp

void cfnConvertLastErrorToString(LPSTR szDest, int nMaxStrLen);
// เป็นฟังก์ชันเปลี่ยน Error ให้ออกมาเป็น String
// อยู่ในไฟล์ \VRCampus\common\vrcommon.cpp

void DisplayPlayers( HWND hDlg ); // เป็นฟังก์ชันแสดงรายชื่อผู้ใช้ ออกมาทาง List
Control
// อยู่ในไฟล์ \VRCampus\ VRServer\vrserver.cpp

INT_PTR CALLBACK DlgServerProc( HWND hDlg, UINT msg,
WPARAM wParam, LPARAM lParam );
// เป็นฟังก์ชันหลักของไอคอนล็อกช้อยในการรับ
message จาก ระบบปฏิบัติการ Windows
// อยู่ในไฟล์ \VRCampus\VRServer\vrserver.cpp

HRESULT fnLoadMapInfo(); // เป็นฟังก์ชันที่ใช้ในการโหลดข้อมูล Map ต่างๆ
จากฐานข้อมูลตาราง Map เข้าสู่ตัวแปรพอยน์เตอร์
ของ Cmap
// อยู่ในไฟล์ \VRCampus\VRServer\vrserver.cpp

HRESULT fnMailStep1( APP_PLAYER_INFO* pPlayerInfo,char* szToMail );
// เป็นฟังก์ชันส่งเบอร์อีเมลล์ปลายทางไปยังผู้ใช้
เลือกอีเมลล์ปลายทางที่ต้องการจะเมลล์
// อยู่ในไฟล์ \VRCampus\VRServer\vrserver.cpp

HRESULT fnNpcTalk( APP_PLAYER_INFO* pPlayerInfo,DPNID dpnidTarget,
PK_REQUEST_NPCTALK* pNpc );

```

```
// เป็นฟังก์ชันที่ตรวจสอบว่า NPC ที่ผู้ใช้คลิกขวา
// นั้น เป็น NPC ชนิดใด จากนั้นจึงส่งข้อมูล NPC
// นั้นผ่านฟังก์ชัน SendNpcTalkToPlayer
// อยู่ในไฟล์ \VRCampus\VRServer\vrserver.cpp
```

```
HRESULT fnSendMail( APP_PLAYER_INFO* pPlayerInfo, PK_MAIL_STEP2* pMsg);
// เป็นฟังก์ชันส่งเมลไปยัง SMTP Server โดยนำ
// ข้อความที่จะส่ง จากแพ็คเกจที่ได้รับมาจากผู้ใช้
// อยู่ในไฟล์ \VRCampus\VRServer\vrserver.cpp
```

```
HRESULT SendChatMessage( DPNID dpnidFrom, char* szChatMessage, char*
szRecieveName, UINT nMapcode );
// เป็นฟังก์ชันส่งข้อความ chat หาก
szRecieveName มีค่าเป็น "All" คือเป็นการส่งไป
ให้ผู้ใช้ทุกเครื่อง หากมีค่าเป็นชื่อของผู้ใช้คนอื่น ก็จะ
ส่งไปให้เฉพาะผู้ใช้นั้นๆ
// อยู่ในไฟล์ \VRCampus\VRServer\vrserver.cpp
```

```
HRESULT SendCreatePlayerMsg( APP_PLAYER_INFO* pPlayerAbout, DPNID
dpnidTarget );
// เป็นฟังก์ชันส่งข้อมูลไปยังผู้ใช้ทุกเครื่องให้สร้าง
ตัวละครที่เพิ่งเข้ามาใหม่ขึ้นมา
// อยู่ในไฟล์ \VRCampus\VRServer\vrserver.cpp
```

```
HRESULT SendDestroyPlayerMsg(APP_PLAYER_INFO* pPlayerInfo,DPNID dpnidTarget);
// เป็นฟังก์ชันส่งข้อมูลไปยังผู้ใช้ทุกเครื่องให้ลบ
ตัวละครที่มี DPNID เป็น dpnidTarget
// อยู่ในไฟล์ \VRCampus\VRServer\vrserver.cpp
```

```
HRESULT SendLoginToPlayer( DPNID dpnidTarget, int nType );
// เป็นฟังก์ชันส่งข้อมูลตอบกลับการล็อกอินจาก
// ผู้ใช้ว่าล็อกอินสำเร็จหรือไม่
// อยู่ในไฟล์ \VRCampus\VRServer\vrserver.cpp
```

```
HRESULT SendMapToPlayer(DPNID dpnidTarget, char* szUsername, UINT nMapcode, int
nPosX, int nPosY );
// เป็นฟังก์ชันส่งข้อมูลแผนที่ไปยังผู้ใช้ที่ล็อกอิน
// สำเร็จแล้ว
// อยู่ในไฟล์ \VRCampus\VRServer\vrserver.cpp
```

```

HRESULT SendNpcTalkToPlayer(DPNID dpnidTarget, int nNpcID, int nNextStateID, char*
                                szGreeting, char* szCase );
// เป็นฟังก์ชันส่งข้อมูลของ NPC ไปยังผู้ใช้ที่คลิก
// เลือก NPC นั้นๆ
// อยู่ในไฟล์ \VRCampus\VRServer\vrserver.cpp

```

```

HRESULT SendNpcToPlayer(DPNID dpnidTarget, UINT nMapcode );
// เป็นฟังก์ชันส่งข้อมูลของ NPC ทุกตัวไปยังผู้ใช้
// ที่ล็อกอินสำเร็จแล้ว
// อยู่ในไฟล์ \VRCampus\VRServer\vrserver.cpp

```

```

HRESULT SendPlayerInfoToPlayer(DPNID dpnidTarget, char* szUsername );
// เป็นฟังก์ชันส่งข้อมูลส่วนตัวของผู้ใช้ที่ล็อกอิน
// สำเร็จแล้ว ส่งไปให้ผู้ใช้เอง เพื่อให้เห็นตำแหน่ง
// ที่ผู้ใช้ยืนอยู่ตำแหน่งสุดท้ายที่เก็บในฐานข้อมูลได้
// อยู่ในไฟล์ \VRCampus\VRServer\vrserver.cpp

```

```

HRESULT SendWalkMessageToMap( DPNID dpnidFrom, char* szUsername, int
                                nFrameAngle, int xDes, int yDes, int xPos, int
                                yPos, UINT nMapcode );
// เป็นฟังก์ชันที่ส่งการเดินของผู้ใช้ ไปยังผู้ใช้อื่น
// อยู่ในไฟล์ \VRCampus\VRServer\vrserver.cpp

```

```

HRESULT SendWorldStateToNewPlayer( APP_PLAYER_INFO* pPlayerInfo, DPNID
                                dpnidNewPlayer );
// เป็นฟังก์ชันที่ส่ง DPNID ใหม่ ไปยังผู้ใช้ที่
// ล็อกอินสำเร็จแล้ว
// อยู่ในไฟล์ \VRCampus\VRServer\vrserver.cpp

```

```

HRESULT SendWrapToPlayer(DPNID dpnidTarget, UINT nMapcode );
// เป็นฟังก์ชันที่ส่งข้อมูลตำแหน่งวาปทั้งหมดไป
// ยังผู้ใช้ที่ล็อกอินสำเร็จแล้ว
// อยู่ในไฟล์ \VRCampus\VRServer\vrserver.cpp

```

```

HRESULT WINAPI ServerMessageHandler( PVOID pvUserContext, DWORD dwMessageId,
                                PVOID pMsgBuffer );
// เป็นฟังก์ชันเปรียบได้กับ DlgProc ที่รับ message
// ของ Windows แต่ตัวนี้คอยรับ message ของ
// DirectPlay เอง ว่าเป็นการรับหรือส่ง message

```


	// อยู่ในไฟล์ \VRCampus\VRServer\vrserver.cpp
HRESULT StartServer(HWND hDlg);	// เป็นฟังก์ชันที่สั่ง ให้ DirectPlay Initial และ Enum session ให้ Server ทำงาน
	// อยู่ในไฟล์ \VRCampus\VRServer\vrserver.cpp
VOID StopServer(HWND hDlg);	// เป็นฟังก์ชันที่สั่งให้ DirectPlay Cleanup Session ต่างๆ เพื่อปิด Server
	// อยู่ในไฟล์ \VRCampus\VRServer\vrserver.cpp
INT WINAPI WinMain(HINSTANCE hInst, HINSTANCE hPrevInst, LPSTR pCmdLine, INT nCmdShow);	// เป็นฟังก์ชันหลักของโปรแกรม
	// อยู่ในไฟล์ \VRCampus\VRServer\vrserver.cpp

2. โปรแกรมฝั่งไคลเอนต์

ในซอร์สโคดของโปรแกรมฝั่งไคลเอนต์ จะมีโครงสร้าง Structer ต่างๆ ดังนี้

struct APP_PLAYER_INFO	// เป็นโครงสร้างที่เก็บข้อมูลของตัวละครผู้ใช้
	// อยู่ในไฟล์ \VRCampus\VRClient\vrclient.cpp
struct DB_PLAYER_INFO	// เป็นโครงสร้างที่เก็บข้อมูลส่วนตัวของผู้ใช้ที่ได้รับจากฐานข้อมูลจากเซิร์ฟเวอร์
	// อยู่ในไฟล์ \VRCampus\Common\vrcommon.h
struct MAIL_INFO	// เป็นโครงสร้างที่เก็บข้อมูลอีเมล
	// อยู่ในไฟล์ \VRCampus\VRClient\vrclient.cpp
struct NPC_INFO	// เป็นโครงสร้างที่เก็บข้อมูลของ NPC
	// อยู่ในไฟล์ \VRCampus\VRClient\vr3d.h
struct NPC_STATE	// เป็นโครงสร้างที่เก็บสถานะที่คุยกับ NPC ไปว่าไปถึงขั้นตอนใด
	// อยู่ในไฟล์ \VRCampus\VRClient\vr3d.h
struct WRAP_INFO	// เป็นโครงสร้างที่เก็บข้อมูลจุดวาปต่างๆ
	// อยู่ในไฟล์ \VRCampus\VRClient\vr3d.h
struct VERTEXACTOR	// เป็นโครงสร้างที่เก็บเวอร์เท็กซ์ของตัวละคร และ ตำแหน่ง Texture
	// อยู่ในไฟล์ \VRCampus\VRClient\vr3d.h
struct VERTEXMODEL	// เป็นโครงสร้างที่เก็บเวอร์เท็กซ์ของไฟล์แผนที่
	*.X

	// อยู่ในไฟล์ \VRCampus\VRClient\vr3d.h
struct VERTEXPOINTER	// เป็นโครงสร้างที่เก็บเวอร์เท็กซ์ ของตัวชี้ที่พื้น ที่ อยู่ติดตามเมาส์
	// อยู่ในไฟล์ \VRCampus\VRClient\vr3d.h
struct VERTEXSCREEN	// เป็นโครงสร้างที่เก็บเวอร์เท็กซ์ แบบ D3DVERTEX4 ซึ่งจะเป็นตำแหน่งที่เป็น Screen โดยนำมาใช้กับ Minimap
	// อยู่ในไฟล์ \VRCampus\VRClient\vr3d.h
struct VERTEXSPRITE	// เป็นโครงสร้างที่ยังไม่ได้ใช้งานในขณะนี้ มี โครงสร้างคล้าย VERTEXACTOR เพียงแต่มีตัว แปร color เพิ่มขึ้นมา ใช้งานในอนาคตที่มีการ กำหนดแสงเงา
	// อยู่ในไฟล์ \VRCampus\VRClient\vr3d.h

ในซอร์สโคดของโปรแกรมฝั่งไคลเอนท์ ประกอบด้วย class ต่างๆ ดังนี้

class CD3DCamera	// เป็นคลาสที่เกี่ยวกับการกำหนดมุมมองกล้องใน โลก 3 มิติ
	// อยู่ในไฟล์ \VRCampus\VRClient\vr3d.h และ \VRCampus\VRClient\vr3d.cpp
class CD3DMap	// เป็นคลาสที่เกี่ยวกับแสดงผลแผนที่
	// อยู่ในไฟล์ \VRCampus\VRClient\vr3d.h และ \VRCampus\VRClient\vr3d.cpp
class CD3DMapGat	// เป็นคลาสที่เกี่ยวกับการกำหนดระดับพื้นที่
	// อยู่ในไฟล์ \VRCampus\VRClient\vr3d.h และ \VRCampus\VRClient\vr3d.cpp
class CD3DMinimap	// เป็นคลาสที่เกี่ยวกับการแสดงผลแผนที่เล็ก
	// อยู่ในไฟล์ \VRCampus\VRClient\vr3d.h และ \VRCampus\VRClient\vr3d.cpp
class CD3DNpc	// เป็นคลาสที่เกี่ยวกับการแสดงผล NPC ต่างๆ
	// อยู่ในไฟล์ \VRCampus\VRClient\vr3d.h และ \VRCampus\VRClient\vr3d.cpp
class CD3DPointer	// เป็นคลาสที่เกี่ยวกับการแสดงผลของรูปจุดที่พื้น ณ ตำแหน่งเมาส์ชี้

	// อยู่ในไฟล์ \VRCampus\VRClient\vr3d.h และ \\VRCampus\VRClient\vr3d.cpp
class CD3DWrap	// เป็นคลาสที่เกี่ยวกับการแสดงผลความต่าง // อยู่ในไฟล์ \VRCampus\VRClient\vr3d.h และ \\VRCampus\VRClient\vr3d.cpp
class CVRDialogBox	// เป็นคลาสที่เกี่ยวกับการแสดงผลไดอะล็อกบ็อกซ์ // ออกซ์ ต่างๆ เช่น ไดอะล็อกซ์ที่ใช้ในการส่งเมล, ไดอะล็อกซ์แสดงข้อมูลจาก NPC // อยู่ในไฟล์ \VRCampus\VRClient\vrclient.h และ \\VRCampus\VRClient\vrclient.cpp
class CVRLayerWnd	// เป็นคลาสที่ทำให้ไดอะล็อกซ์สามารถแสดงผล แบบ Transparent ได้ // อยู่ในไฟล์ \VRCampus\Common\vrcommon.h และ \\VRCampus\Common\vrcommon.cpp
class CVRMultilineText	// เป็นคลาสที่ทำหน้าที่ เก็บบันทึกค่าให้ Text Control แสดงข้อมูล Text ออกมาแบบ Multiline โดยจะมีการกำหนด MaxLine ของ Multiline ที่จะ แสดง เพื่อให้เมื่อมีการรันแสดงข้อมูล Text มากๆ จะไม่แสดงทั้งหมด จะแสดงเพียงบรรทัด MaxLine หลังสุดเท่านั้น // อยู่ในไฟล์ \VRCampus\Common\vrcommon.h และ \\VRCampus\Common\vrcommon.cpp
class CVRUser	// เป็นคลาสที่เกี่ยวกับข้อมูลผู้ใช้ที่ใช้โคลเอ็นท์ เครื่องนั้นๆ // อยู่ในไฟล์ \VRCampus\VRClient\vrclient.h และ \\VRCampus\VRClient\vrclient.cpp
class CMyD3DApplication : public CD3DApplication	// เป็นคลาสที่สืบทอดมาจาก CD3DApplication ซึ่งเป็นคลาสที่มีคุณสมบัติในการใช้งาน Direct3D และมีการเพิ่ม member variable object ของ Direct3D, DirectInput, DirectPlay เพิ่มในคลาสนี้ ด้วย

// อยู่ในไฟล์ \VRCampus\VRClient\vrclient.h
และ \VRCampus\VRClient\vrclient.cpp

ในซอร์สโค้ดของโปรแกรมฝั่งเซิร์ฟเวอร์ประกอบด้วย ฟังก์ชันหลักๆ แบบ Global ซึ่งจะขออธิบายเพียงบางฟังก์ชันหลัก ดังนี้

```
VOID PlayerAddStruct( APP_PLAYER_INFO* pPlayerInfo );
```

// ฟังก์ชันนี้ทำงานเมื่อมีผู้ใช้เข้ามาใหม่ จะมีการ
เพิ่มข้อมูลของผู้ใช้เข้าไปใน link list ของรายการ
ผู้ใช้ทั้งหมด

// อยู่ในไฟล์ \VRCampus\VRClient\vrclient.cpp

```
VOID PlayerDestoryAll();
```

// ฟังก์ชันนี้เป็นการทำลาย ทุกรายการของผู้ใช้
ทั้งหมด

// อยู่ในไฟล์ \VRCampus\VRClient\vrclient.cpp

```
VOID PlayerDestoryStruct( APP_PLAYER_INFO* pPlayerInfo );
```

// ฟังก์ชันนี้เป็นการทำลาย ข้อมูลของผู้ใช้
pPlayerInfo

// อยู่ในไฟล์ \VRCampus\VRClient\vrclient.cpp

```
HRESULT PlayerGetStruct( DPNID dpnidPlayer, int xPos, int yPos, APP_PLAYER_INFO**  
ppPlayerInfo );
```

// ฟังก์ชันนี้ดึงข้อมูลผู้ใช้ ppPlayerInfo จากรายการ
link list ของผู้ใช้

// อยู่ในไฟล์ \VRCampus\VRClient\vrclient.cpp

```
VOID PlayerSetChat( DPNID dpnidPlayer, char* szChatMessage, HWND hDlg );
```

// ฟังก์ชันนี้ทำงานเมื่อผู้ใช้พิมพ์ข้อความส่ง chat
ก่อนที่แป้นกด chat จะถูกส่งออกไปนั้น ฟังก์ชันนี้
จะทำการกำหนดข้อความ chat และ DPNID ของ
ผู้ใช้ที่ทำการส่ง

// อยู่ในไฟล์ \VRCampus\VRClient\vrclient.cpp

```
VOID PlayerSetFrame( FLOAT fCamAngle );
```

// ฟังก์ชันนี้ใช้กำหนด Frame ของภาพ bitmap
ของตัวละคร เมื่อหมุนไปยังมุมมองต่างๆ รวมถึง
การกำหนดเปลี่ยน Frame เมื่อมีการเดินของตัว
ละครด้วย

// อยู่ในไฟล์ \VRCampus\VRClient\vrclient.cpp

```
VOID PlayerSetWalk( DPNID dpnidPlayer, INT nFrameAngle, INT xPos, INT yPos, INT
xDes, INT yDes );
```

// ฟังก์ชันนี้ใช้กำหนดการเดินของตัวละคร

// อยู่ในไฟล์ \VRCampus\VRClient\vrclient.cpp

3. แพ็กเก็ตที่ใช้ในการรับส่งข้อมูลระหว่างเซิร์ฟเวอร์กับไคลเอนต์

```
struct PK_GENERIC
```

```
{ WORD wID; };
```

ในทุกแพ็กเก็ตจะมีข้อมูล 2 ไบต์แรกเป็นข้อมูลส่วนหัวของส่วนดาต้าในแพ็กเก็ต โดยจะมีการกำหนดเป็นลักษณะโครงสร้าง PK_GENERIC แบบโคดข้างบนนี้ ซึ่งจะมีการกำหนดค่าให้กับ wID เพื่อระบุว่าแพ็กเก็ตนี้เป็นแพ็กเก็ตเพื่อใช้งานในคำสั่งใด มีหมายเลขคำสั่ง ดังนี้

#define PK_ID_RESPONSE_LOGIN	1
#define PK_ID_SET_ID	3
#define PK_ID_CREATE_PLAYER	4
#define PK_ID_DESTROY_PLAYER	5
#define PK_ID_PLAYER_WALK	6
#define PK_ID_PLAYER_CHAT	7
#define PK_ID_REQUEST_MAP	11
#define PK_ID_RESPONSE_MAP	12
#define PK_ID_RESPONSE_WRAP	14
#define PK_ID_RESPONSE_NPC	16
#define PK_ID_REQUEST_NPCTALK	17
#define PK_ID_RESPONSE_NPCTALK	18
#define PK_ID_MAIL_STEP1	19
#define PK_ID_MAIL_STEP2	20
#define PK_ID_REQUEST_PLAYERINFO	21
#define PK_ID_RESPONSE_PLAYERINFO	22

ตารางที่ 1 แสดงหมายเลขคำสั่งต่างๆ

สำหรับในแต่ละคำสั่งนั้น จะมีโครงสร้างแพ็กเก็ตต่างๆ กันไป ดังนี้

PK_ID_RESPONSE_LOGIN :	
<pre> struct PK_RESPONSE_LOGIN : public PK_GENERIC { int nReturn; }; </pre>	เป็นโครงสร้างแพ็คเกจที่ใช้ตอบกลับการล็อกอินจากผู้ใช้งานถูกต้องหรือไม่ ลักษณะการส่ง : เซิร์ฟเวอร์จะส่งข้อมูลหรือข้อผิดพลาดในการล็อกอินไปให้ไคลเอ็นท์

ตารางที่ 2 แสดงโครงสร้างแพ็คเกจ PK_RESPONSE_LOGIN

PK_ID_SET_ID : 3	
<pre> struct PK_SET_ID : public PK_GENERIC { DWORD dpnidPlayer; UINT nMapcode; CHAR szMapname[MAX_CHAR]; INT xPos; INT yPos; }; </pre>	เป็นโครงสร้างแพ็คเกจที่เป็นคำสั่งในการกำหนดค่าให้กับตัวละครผู้ใช้ หลังจากที่ผู้ใช้ล็อกอินเรียบร้อยแล้ว จะส่งข้อมูลรหัสของตัวละครในการเชื่อมต่อ รหัสและชื่อแผนที่ รวมถึงตำแหน่งที่ตัวละครอยู่เป็นครั้งสุดท้าย ลักษณะการส่ง : เซิร์ฟเวอร์จะส่งไปให้เฉพาะผู้ใช้ที่ล็อกอินเข้ามา

ตารางที่ 3 แสดงโครงสร้างแพ็คเกจ PK_SET_ID

PK_ID_CREATE_PLAYER : 4	
<pre> struct PK_CREATE_PLAYER : public PK_GENERIC { DWORD dpnidPlayer; CHAR szUsername[MAX_CHAR]; UINT nMapcode; INT xPos; INT yPos; }; </pre>	เป็นโครงสร้างแพ็คเกจที่เป็นคำสั่งในการแสดงตัวละครของผู้ใช้ที่ล็อกอินเข้ามาใหม่ เพื่อให้โปรแกรมไคลเอ็นท์สร้างและแสดงตัวละครขึ้นมาให้เห็นในตำแหน่งที่ระบุ ทำให้ทุกไคลเอ็นท์มองเห็นตัวละครใหม่ได้เหมือนกันหมด ลักษณะการส่ง : เซิร์ฟเวอร์จะส่งไปให้ผู้ใช้ทุกคน

ตารางที่ 4 แสดงโครงสร้างแพ็คเกจ PK_CREATE_PLAYER

PK_ID_DESTROY_PLAYER : 5	
<pre> struct PK_DESTROY_PLAYER : public PK_GENERIC { DWORD dpnidPlayer; }; </pre>	เป็นโครงสร้างแพ็กเก็ตที่เป็นคำสั่งในการลบตัวละคร ออกจากโปรแกรมฝั่งไคลเอ็นท์ ลักษณะการส่ง : เซิร์ฟเวอร์จะส่งหมายเลขของผู้ใช้ในการเชื่อมต่อ นั้น ไปให้ทุกไคลเอ็นท์

ตารางที่ 5 แสดงโครงสร้างแพ็กเก็ต PK_DESTROY_PLAYER

PK_ID_PLAYER_WALK : 6	
<pre> struct PK_PLAYER_WALK : public PK_GENERIC { DWORD dpnidPlayer; INT nFrameAngle; INT xPos; INT yPos; INT xDes; INT yDes; }; </pre>	เป็นโครงสร้างแพ็กเก็ตที่เป็นคำสั่งในการเดินของตัวละคร ลักษณะการส่ง : ฝั่งไคลเอ็นท์จะส่งแพ็กเก็ตนี้ไปบอกเซิร์ฟเวอร์ว่า ต้องการเดิน เซิร์ฟเวอร์จะส่งข้อมูลกลับไปยังไคลเอ็นท์ทุกเครื่อง ว่า มีผู้ใช้กำลังเดินอยู่

ตารางที่ 6 แสดงโครงสร้างแพ็กเก็ต PK_PLAYER_WALK

PK_ID_PLAYER_CHAT : 7	
<pre> struct PK_PLAYER_CHAT : public PK_GENERIC { DWORD dpnidPlayer; CHAR szChatMessage[MAX_PATH]; CHAR szReceiveName[MAX_CHAR]; UINT nMapcode; }; </pre>	เป็นโครงสร้างแพ็กเก็ตที่เป็นคำสั่งในการสนทนาของตัวละคร ลักษณะการส่ง : ถ้าเป็นการสนทนาแบบสาธารณะ ฝั่งไคลเอ็นท์จะส่งแพ็กเก็ตนี้ไปบอกเซิร์ฟเวอร์ว่า ต้องการสนทนา เซิร์ฟเวอร์จะส่งข้อมูลกลับไปยังไคลเอ็นท์ทุกเครื่อง ว่า มีข้อความสนทนาเกิดขึ้น ถ้าเป็นการสนทนาแบบกระซิบ เซิร์ฟเวอร์จะตรวจสอบชื่อผู้ใช้ที่เป็นเป้าหมายที่จะสนทนาด้วย ว่ามีรายชื่อออนไลน์อยู่หรือไม่ แล้วทำการส่งไปยังผู้ใช้นั้น กับผู้ใช้ที่เป็นผู้ส่ง

ตารางที่ 7 แสดงโครงสร้างแพ็กเก็ต PK_PLAYER_CHAT

PK_ID_REQUEST_MAP : 11	
<pre> struct PK_REQUEST_MAP : public PK_GENERIC { UINT nMapcode; int nPosX; int nPosY; }; </pre>	เป็นโครงสร้างแฟกเก็ตที่เป็นคำสั่งในการร้องขอรหัสแผนที่ใหม่ที่ผู้ใช้จะไป เกิดขึ้นเมื่อผู้ใช้เดินไปถึงจุดเชื่อมต่อระหว่างแผนที่ ลักษณะการส่ง : โคลเอ็นท์จะส่งแฟกเก็ตไปบอกตำแหน่งและแผนที่เดิมที่ผู้ใช้อยู่ก่อนจะทำการเปลี่ยนแผนที่

ตารางที่ 8 แสดงโครงสร้างแฟกเก็ต PK_REQUEST_MAP

PK_ID_RESPONSE_MAP : 12	
<pre> struct PK_RESPONSE_MAP : public PK_GENERIC { CHAR szMapname[MAX_CHAR]; int nPosX; int nPosY; }; </pre>	เป็นโครงสร้างแฟกเก็ตที่เป็นคำสั่งในการตอบกลับรหัสแผนที่ใหม่ ที่ผู้ใช้ที่ร้องขอมา ลักษณะการส่ง : เซิร์ฟเวอร์จะส่งแฟกเก็ตนี้กลับไปบอกข้อมูลรหัสแผนที่และตำแหน่งใหม่ให้โคลเอ็นท์ที่ร้องขอมาด้วยแฟกเก็ต PK_REQUEST_MAP

ตารางที่ 9 แสดงโครงสร้างแฟกเก็ต PK_RESPONSE_MAP

PK_ID_RESPONSE_WRAP : 14	
<pre> struct PK_RESPONSE_WRAP : public PK_GENERIC { UINT nSrcMapCode; int nSrcX; int nSrcY; UINT nDestMapCode; int nDestX; int nDestY; }; </pre>	เป็นโครงสร้างแฟกเก็ตที่เป็นคำสั่งในการบอกข้อมูลจุดเชื่อมต่อใหม่ในแผนที่ใหม่ให้กับผู้ใช้ที่ทำการเปลี่ยนแผนที่ ลักษณะการส่ง : เซิร์ฟเวอร์จะส่งแฟกเก็ตนี้กลับไปบอกข้อมูลจุดเชื่อมต่อใหม่ในแผนที่ใหม่ให้โคลเอ็นท์ที่ร้องขอมาด้วยแฟกเก็ต PK_REQUEST_MAP

ตารางที่ 10 แสดงโครงสร้างแฟกเก็ต PK_RESPONSE_WRAP

PK_ID_RESPONSE_NPC : 16	
<pre> struct PK_RESPONSE_NPC : public PK_GENERIC { int nNpcID; UINT nMapCode; int x; int y; int nBmpID; int nFuncID; char szGreeting00[256]; char szGreeting01[256]; char szGreeting02[256]; }; </pre>	เป็นโครงสร้างแพ็กเก็ตที่เป็นคำสั่งในการบอกข้อมูลตัวละครจำลองพิเศษ (Non-Plyer Character : NPC) ในแผนที่ใหม่ให้กับผู้ใช้ที่ทำการเปลี่ยนแผนที่ ลักษณะการส่ง : เซิร์ฟเวอร์จะส่งแพ็กเก็ตนี้กลับไปบอกข้อมูลตัวละครจำลองพิเศษในแผนที่ใหม่ให้ไคลเอ็นท์ที่ร้องขอมาด้วยแพ็กเก็ต PK_REQUEST_MAP

ตารางที่ 11 แสดงโครงสร้างแพ็กเก็ต PK_RESPONSE_NPC

PK_ID_REQUEST_NPCTALK : 17	
<pre> struct PK_REQUEST_NPCTALK : public PK_GENERIC { int nNpcID; int nTalkID; int nCaseID; int nStateID; }; </pre>	เป็นโครงสร้างแพ็กเก็ตที่เป็นคำสั่งในการร้องขอข้อมูลการใช้งานของตัวละครจำลองพิเศษ (Non-Plyer Character : NPC) ที่ผู้ใช้ได้กดคลิกขวาเลือกไว้ ลักษณะการส่ง : ไคลเอ็นท์จะส่งแพ็กเก็ตนี้ไปเพื่อขอข้อมูลการใช้งานของตัวละครจำลองพิเศษจากเซิร์ฟเวอร์

ตารางที่ 12 แสดงโครงสร้างแพ็กเก็ต PK_REQUEST_NPCTALK

PK_ID_RESPONSE_NPCTALK : 18	
<pre> struct PK_RESPONSE_NPCTALK : public PK_GENERIC { int nNpcID; int nNextStateID; char szGreeting[256]; char szCase[256]; }; </pre>	เป็นโครงสร้างแฟกเก็ตที่เป็นคำสั่งในการตอบกลับข้อมูลการใช้งานของตัวละครจำลองพิเศษ (Non-Player Character : NPC) ที่ผู้ใช้ได้กดคลิกขวาเลือกไว้ ลักษณะการส่ง : เซิร์ฟเวอร์จะส่งแฟกเก็ตนี้ไปเพื่อบอกข้อมูลการใช้งานของตัวละครจำลองพิเศษที่ไคลเอ็นท์ต้องการ

ตารางที่ 13 แสดงโครงสร้างแฟกเก็ต PK_RESPONSE_NPCTALK

PK_ID_MAIL_STEP1 : 19	
<pre> struct PK_MAIL_STEP1 : public PK_GENERIC { char szToMail[30]; }; </pre>	เป็นโครงสร้างแฟกเก็ตที่เป็นคำสั่งในการบอกเบอร์อีเมลปลายทาง ลักษณะการส่ง : เซิร์ฟเวอร์จะส่งแฟกเก็ตนี้ไปให้ไคลเอ็นท์ เพื่อบอกเบอร์อีเมลปลายทาง

ตารางที่ 14 แสดงโครงสร้างแฟกเก็ต PK_MAIL_STEP1

PK_ID_MAIL_STEP2 : 20	
<pre> struct PK_MAIL_STEP2 : public PK_GENERIC { char szToMail[30]; char szSubject[30]; char szBody[512]; }; </pre>	เป็นโครงสร้างแฟกเก็ตที่เป็นคำสั่งในการส่งข้อมูลเนื้อหาของเมล ลักษณะการส่ง : ไคลเอ็นท์จะส่งแฟกเก็ตนี้ไปให้เซิร์ฟเวอร์ทำการส่งเมลไปยังเบอร์อีเมลปลายทาง

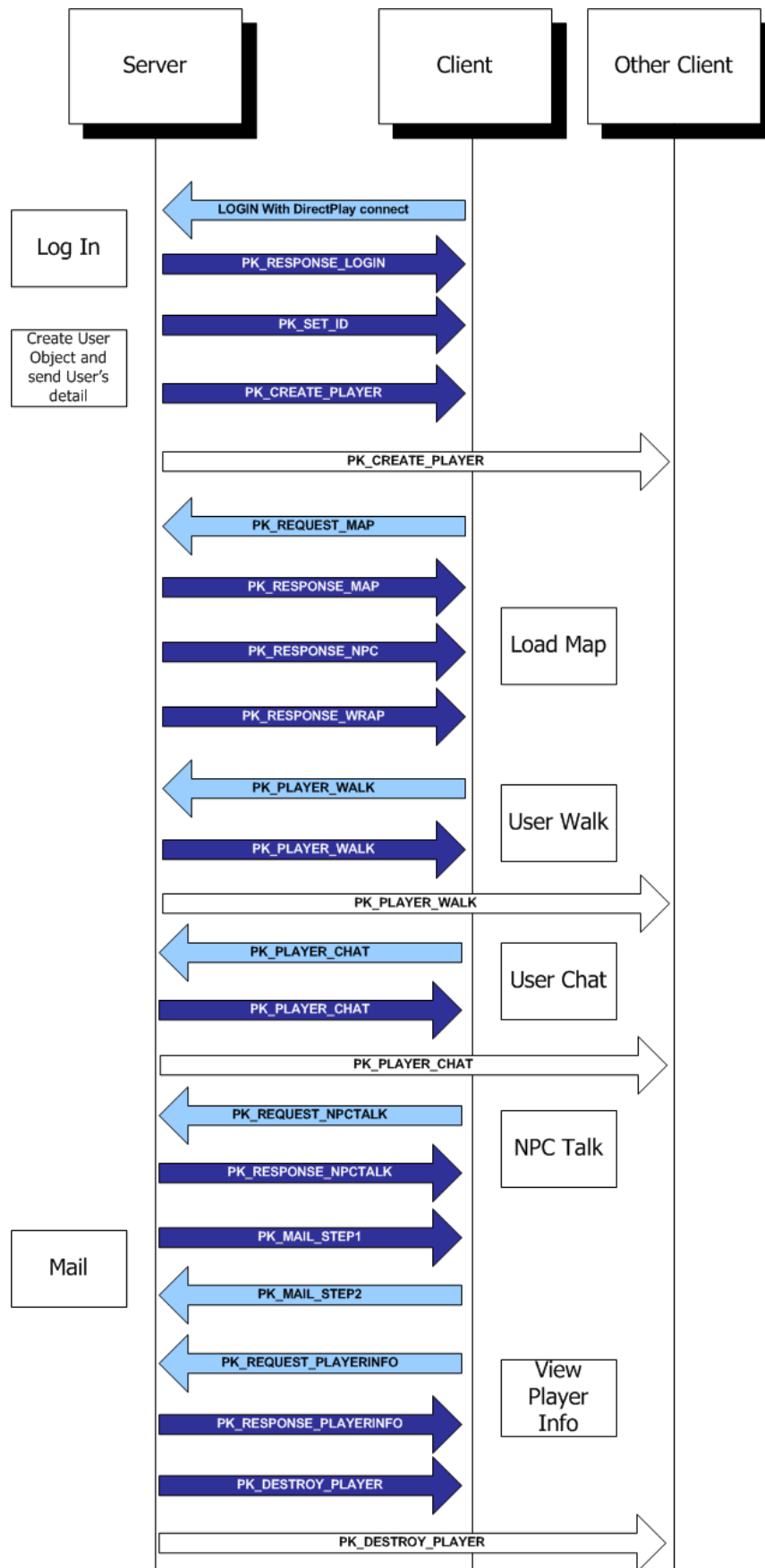
ตารางที่ 15 แสดงโครงสร้างแฟกเก็ต PK_MAIL_STEP2

PK_ID_REQUEST_PLAYERINFO : 21	
<pre> struct PK_REQUEST_PLAYERINFO : public PK_GENERIC { char szUsername[12]; }; </pre>	เป็นโครงสร้างแพ็กเก็ตที่เป็นคำสั่งในการร้องขอข้อมูลรายละเอียดของผู้ใช้ที่ถูกเลือกดูรายละเอียด ลักษณะการส่ง : ไคลเอนท์จะส่งแพ็กเก็ตนี้ไปให้เซิร์ฟเวอร์ เป็นการร้องขอรายละเอียดของผู้ใช้ที่ไคลเอนท์เลือกดูข้อมูล

ตารางที่ 16 แสดงโครงสร้างแพ็กเก็ต PK_REQUEST_PLAYERINFO

PK_ID_RESPONSE_PLAYERINFO : 22	
<pre> struct PK_RESPONSE_PLAYERINFO : public PK_GENERIC { DB_PLAYER_INFO dbPlayerInfo; }; </pre>	เป็นโครงสร้างแพ็กเก็ตที่เป็นคำสั่งในการตอบกลับด้วยข้อมูลรายละเอียดของผู้ใช้ที่ถูกเลือกดูรายละเอียด ลักษณะการส่ง : เซิร์ฟเวอร์จะส่งแพ็กเก็ตนี้ไปให้ไคลเอนท์ ที่ร้องขอรายละเอียด

ตารางที่ 17 แสดงโครงสร้างแพ็กเก็ต PK_RESPONSE_PLAYERINFO



รูปที่ 8 แสดงแพ็กเก็ตที่ส่งกลับไปมาระหว่างไคลเอนต์กับเซิร์ฟเวอร์